
Rapport

**POLYTECHNIQUE
MONTRÉAL**

UNIVERSITÉ
D'INGÉNIERIE



Prototypage d'une application **Fractale**

ELE8307 – Prototypage rapide de systèmes numériques

Travail présenté à Jean-Pierre David et Mengyue Su

Hiver 2021

Département de génie électrique
École Polytechnique de Montréal

Dernière mise à jour : 22 mars 2021 à 07:56:31

Michael **Flandez**

1825018

Sekou **Kone**

1809202

1 Introduction

Ce document représente le rapport d'étape de notre projet dans le cadre du cours ELE8307 - Prototypage rapide de systèmes numériques. Ce projet consiste en l'accélération d'une application qui effectue plusieurs opérations mathématiques pour produire la représentation d'une fractale. La fractale est obtenue en étudiant la divergence sur 64 itérations de la suite $Z_{n+1} = Z_n^2 + C$. On admet qu'il y a divergence lorsque le module du nombre complexe Z_{n+1} est supérieur à 2.

Pour accélérer cette application, dans un premier temps, on se concentrera sur le code en C et on mettra en pratique les diverses techniques d'optimisation étudiées jusqu'à présent. Par la suite, on identifiera certaines parties critiques du code qu'on transformera en machines à état algorithmiques pour une implémentation matérielle. Rendus au terme de ces améliorations, on sera en possession d'une application pouvant produire un nombre acceptable d'images par secondes

2 Analyse des temps et partage de l'application logiciel/matériel

Dans le processus d'évolution d'une solution technique logiciel on se retrouve souvent avec des inconvénients inattendus. Au début on a un besoin qu'amènera vers la conception d'une solution. Durant la conception, on teste des différentes configurations ou architectures possibles théoriquement pour ensuite choisir la plus optimale à implémenter en prototype. Une fois le prototype créé, on passe aux tests. Alors c'est ici où on vérifie le bon fonctionnement de la logique, la robustesse des algorithmes et la vitesse des calculs ou performance. Si les résultats ne sont acceptables et le budget le permet, on débogue et trouve des moyens pour améliorer le projet.

Le solution mise à notre disposition est prête à être utilisé en Visual Studio. Lorsqu'on lance l'application, on voit bien la production des fractals et de dix perles dans un espace de deux dimensions. L'application utilise la librairie **OpenGL** compte sur plusieurs sous routines pour dessiner des objets sur un écran d'une taille de 480 * 640 pixels. Ensuite on a implémenté la même solution dans un projet **Quartus** pour l'implémenter sur la carte programmable DE-10. Cela étant, on a ouvert un nouveau projet sur l'IDE **Eclipse** contenant les mêmes fichiers de source et d'en tête de la solution originale, mais cette fois-ci, avec l'option POINT_FIXE. Le choix de POINT_FIXE est choisi parce qu'il a une meilleure résolution par rapport à la représentation en virgule flottant. En plus, ce choix facilitera les calculs et optimisations implémentés en VHDL.

En faisant plusieurs rapports de profilage avec des différents valeurs initiales, on a pu voir le comportement général de l'application. Le tableau 1 affichera les différents appels à des routines et sous routines trouvés avec Eclipse.

Remarque importante

Si bien les rapports de logiciel peuvent nous indiquer les emplacements de plus grande demande des ressources dans de l'application, ils sont limités et ne voient pas tout ce qui se passe en vérité. Dans notre cas, lorsqu'on s'approche aux perles dans l'application et ils deviennent plus grands, on voit sur l'écran que l'application prends plus de temps pour calculer et dessiner les pixels. Les perles ont l'air de clignoter. Ce phénomène n'est pas mentionné dans les rapports de profilage sur Eclipse.

2.1 Les temps les plus significatifs

				% du temps total	temps d'exécution	# d'appels du parent
main	draw_fractal()			4.47	3.19	1
		Get_Fractal_Level()		24.61	17.55	306081
		__muldi3		6.97	4.97	12529465
		__fixdfdi		0.85	0.60	612162
		__fixunsdfdi		0.43	0.60	306641
		__muldf3		2.73	1.95	612162
		__fixunsdfsi		0.53	0.38	1224332
		__subdf3		1.30	0.93	612442
		__floatunsidf		0.47	0.34	612166
		__ltdi2		0.49	0.35	612286
		__fixunsdfdi		0.43	0.60	306641
		__muldf3		2.73	1.95	612162
		__fixunsdfsi		0.53	0.38	1224332
		__subdf3		1.30	0.93	612442
		__floatunsidf		0.47	0.34	612166
		__muldf3		5.46	3.90	1224324
	__divdf3			6.90	4.92	1224324
	__udivsi3			14.60	10.41	4897700
	__umodsi3			13.85	9.88	4897700
	__muldf3			5.46	3.90	1224324
	__adddf3			2.07	1.48	1224324
	__floatsidf			0.8	0.65	612162
	__clzsi2			0.9	0.65	612467
	VGA::Set_Pixel_Color			0.45	0.32	307200
	get_color(int)			0.24	0.17	306081
	draw_balls()			0.03	0.02	1
	VGA::Set_Pixel_Color			0.95		617016
	draw_ball3D()			0.01		10

Tableau 1 : Arbre de dépendances du rapport de profilage sur Eclipse

On voit sur le tableau 1 un arbre de dépendances extrait des plusieurs analyses. Des analyses avec des itérations de 1, 3 et 10 fois de la fonction principale ont été faits, ainsi qu'en changeant l'échelle de perles de 0.25 à 0.5, 0.75 et 1. Après ces tests on a remarqué des petites variations dans les temps totaux d'exécution, entre 68 et 74 seconds, mais où les pourcentages des temps n'ont vraiment pas changé.

2.2 Influences des diverses fonctions

En examinant l'exemple avec un itérations et l'échelle de la balle à 0.5, on a construit le tableau 1. La fonction main(), dans la première colonne est appelé une seule fois. Main() fait appel à quelques fonctions mais on va juste discuter de celles qui prennent le plus de temps (selon le rapport de profilage en

Rapport – Fractale

Eclipse) qui sont dans la deuxième colonne en couleur jaune. Là on voit que la fonction `draw_fractal()` utilise 4.47% de temps total tandis que `draw_ball()` prends 0.03%. En même temps, `draw_fractal()` utilise d'autres fonctions également affichées sur la troisième colonne, entre-elles `Get_Fractal_Level()` avec le 24.61% , `_divdf` avec 6.9% , etc. `Draw_ball()` a appelé des fonction qui ne requièrent pas beaucoup des ressources et ne sont pas affichés. L'arbre continue vers la droite sur la quatrième, cinquième et sixième colonne.

Les points chauds

En couleur rouge on trouve les fonctions ou opérations qui utilise le plus des ressources. Le plus consommant est `Get_Fractal_level()` qui utilise en tout 24.61%, en deuxième on a `_udivsi` (14.60%) et `_umodsi3` (13.85%). Ces deux sous routines forment partie de `_divdf3`(6.9%) et troisièmement `_muldf3` (5.46%).

D'ici on peut conclure que la fonction `draw_ball()` utilise beaucoup de ressources, spécialement du côté des divisions des double et aussi des multiplications des double. Et on peut aussi conclure que `Get_Fractal_Level()` est très consommant et donc mérite notre attention.

2.3 Le cas des perles proches

Étant donné les limitations du profilage, on sait qu'il existe un problème avec l'affichage des perles même si dans une itération ce n'est pas évident. Il sera dans notre intérêt de trouver une explication à ce problème.

En regardant le tableau 1, on voit que la sous routine `VGA::set_pixel_color` est appelé dans la fonction `draw_fractal()` et prends 0.45 % à chaque appel, qui n'est pas très consommant. Ensuite, au tout bas du tableau 1 on voit le nombre total d'appels fait à `VGA::set_pixel_color` qui est 617016 qui représente un 0.95% du total.

D'un autre côté, un profilage a été fait sur Visual Studio et les résultats sont :

Nom de la fonction	Nombre d'appels	% de temps inclusif écoulé	% de temps exclusif écoulé	Temps inclusif écoulé moy.	Temps exclusif écoulé moy.	Nom du module
<code>VGA::Set_Pixel_Color</code>	4 022 496	86.98 %	46.95 %	0,00	0,00	<code>OpenGLFractal.exe</code>
<code>glVertex2f</code>	4 022 496	13.72 %	13.72 %	0,00	0,00	<code>OPENGL32.dll</code>
<code>glBegin</code>	4 022 496	10.17 %	10.17 %	0,00	0,00	<code>OPENGL32.dll</code>
<code>draw_fractal</code>	12	85.29 %	10.17 %	554,54	66,12	<code>OpenGLFractal.exe</code>
<code>glColor3f</code>	4 022 496	10.11 %	10.11 %	0,00	0,00	<code>OPENGL32.dll</code>
<code>glEnd</code>	4 022 496	6.03 %	6.03 %	0,00	0,00	<code>OPENGL32.dll</code>
<code>glutMainLoop</code>	1	87.19 %	1.00 %	6 802,94	77,95	<code>freeglut.dll</code>
<code>_glutCreateWindow...</code>	1	0.81 %	0.81 %	62,96	62,96	<code>freeglut.dll</code>
<code>init</code>	1	11.98 %	0.32 %	934,79	25,34	<code>OpenGLFractal.exe</code>
<code>display</code>	74 866	86.19 %	0.24 %	0,09	0,00	<code>OpenGLFractal.exe</code>
<code>_stdio_common_vfpr...</code>	50	0.21 %	0.21 %	0,32	0,32	<code>ucrtbase.dll</code>
<code>glutGet</code>	74 891	0.12 %	0.12 %	0,00	0,00	<code>freeglut.dll</code>
<code>glFlush</code>	36	0.10 %	0.10 %	0,22	0,22	<code>OPENGL32.dll</code>
<code>drawBall3D</code>	120	0.32 %	0.03 %	0,21	0,02	<code>OpenGLFractal.exe</code>
<code>_glutInitWithExit</code>	1	0.02 %	0.02 %	1,57	1,57	<code>freeglut.dll</code>
<code>glClear</code>	1	0.00 %	0.00 %	0,10	0,10	<code>OPENGL32.dll</code>

Tableau 2 : Profilage sur Visual Studio, fonctions plus consommant de ressources.

Sur le tableau 2 on voit une liste des fonctions qui consomment le plus des ressources du processeur. Là on voit `VGA::Set_Pixel_Color` en première place utilisant 46.95%, `glVertex2f` et `glBegin` sont des instructions de la librairie `OpenGL` avec 13.72% et 10.17% respectivement, en quatrième place se trouve `draw_fractal()` avec 10.17%.

On peut conclure ici que `VGA::Set_Pixel_Color` utilise beaucoup des ressources.

3 Partage de l'application logiciel/matériel

Il nous est évident en ce moment qu'on doit mettre emphase pour accélérer les deux groupes de fonctions qui affichent sur l'écran. D'un côté, on a `draw_fractal()` qui fait des milliers des calculs (multiplications et divisions) et un appel à `VGA ::Set_Pixel_Color` à chaque fois qu'elle est appelé, en plus de faire appel à la fonction `Get_Fractale_Level()` qui fait des milliers des calculs elle aussi (multiplications). De l'autre côté on a la partie de `draw_balls()` qui fait appel à `draw_balls3d` avec des calculs de racine carrée d'une division en plus de l'utilisation de `VGA ::Set_Pixel_Color` des milliers des fois pour dessiner les perles.

Comme stratégie d'optimisation, on va se concentrer sur ces deux parties par l'optimisation du code C++ et par l'implémentation des instructions spécialisés.

3.1 Optimisation en C++

Premièrement, on s'attaque à la fonction `draw_fractal()`. Pour ce faire, on a calculé deux tableaux, de 480 et 640 éléments pour éviter de calculer une division à chaque itération, ce qui nous sauve la plupart des calculs de divisions en modulus.

Dans la fonction `draw_ball3D()`, on a implémenté une fonction de `sqrt()` plus optimisé trouvé sur le site web [StackOverflow](#)¹.

Dans la fonction `update_position()`, on a modifié le code de manière à réduire le nombre des divisions nécessaires par des précalculs.

On a aussi changé le type des variables des double à des long long pour faciliter l'implémentation des instructions spécialisées sur la carte FPGA.

3.2 Implémentation de deux ASM

Deux instructions spécialisées ont été implémentées. La première est la fonction `Get_Fractale_Level()` au complet et la deuxième est une partie de la fonction `draw_ball3D()`, un appel de la fonction `draw_fractal()`, et un appel de la fonction `VGA ::clr()`, lorsque l'instruction `VGA ::set_pixel_color()` est appelé.

3.2.1 Instruction spécialisée 1 (Sekou Kone)

Code source

```

unsigned int get_fractal_level(int n, unsigned int dataa, unsigned int datab) { // pas optimise unsigned int dataa, unsigned int datab
int start = 1; // Not really used, just for simulation purpose in C/C++
int valid = 0; // Not really used, just for simulation purpose in C/C++
static long long new_Re; //All registers must be declared static to keep their value from call to call
static long long new_Im; //idem
static long long old_Re; //idem
static long long old_Im; //idem
static long long sq_Re; //idem
static long long sq_Im; //idem
static unsigned int fixed_precision; //idem
static long long c_Re_lo, c_Re_hi, c_Im_lo, c_Im_hi; //idem
static long long c_Re, c_Im; //idem
static long long new_Re_lo, new_Re_hi, new_Im_lo, new_Im_hi; //idem
static int i; //idem

INIT: if (start == 0) {valid=0; goto INIT; }
      else if ((start == 1) && (n == 0)) { valid = 0; old_Re = 0; old_Im = 0; sq_Re = 0; sq_Im = 0; i = 0; c_Re = (c_Re_hi << 32) + c_Re_lo;
                                         c_Im = (c_Im_hi << 32) + c_Im_lo; new_Re = (new_Re_hi << 32) + new_Re_lo;
                                         new_Im = (new_Im_hi << 32) + new_Im_lo; goto S1; }
      else if ((start == 1) && (n == 1)) { valid = 1; fixed_precision = dataa; start = 0; return dataa; }
      else if ((start == 1) && (n == 2)) { valid = 1; c_Re_lo = dataa; c_Re_hi = datab; start = 0; return dataa; }
      else if ((start == 1) && (n == 3)) { valid = 1; c_Im_lo = dataa; c_Im_hi = datab; start = 0; return dataa; }
      else if ((start == 1) && (n == 4)) { valid = 1; new_Re_lo = dataa; new_Re_hi = datab; start = 0; return dataa; }
      else if ((start == 1) && (n == 5)) { valid = 1; new_Im_lo = dataa; new_Im_hi = datab; start = 0; return dataa; }
      else { valid = 0; goto INIT; }

S1: if (i < 64) {old_Re = new_Re; old_Im = new_Im; sq_Re = new_Re * new_Re; sq_Im = new_Im * new_Im; goto S12; }
    else {valid = 1; return i; }

S12: if (!) {sq_Re >= fixed_precision; sq_Im >= fixed_precision; goto S2; }

S2: if ((sq_Re + sq_Im) > (4 << fixed_precision)) {valid = 1; return i; }
    else {new_Re = sq_Re - sq_Im + c_Re; new_Im = old_Re * old_Im; goto s22; }

s22: if (!) {new_Im >= (fixed_precision - 1); goto s23; }

s23: if (!) {new_Im = new_Im + c_Im; i++; goto S1; }
}

```

Figure 1 : Code source pour l’instruction multicycle qui prends trois paramètres unsigned int et retourne le resultat au code C++.

Schéma

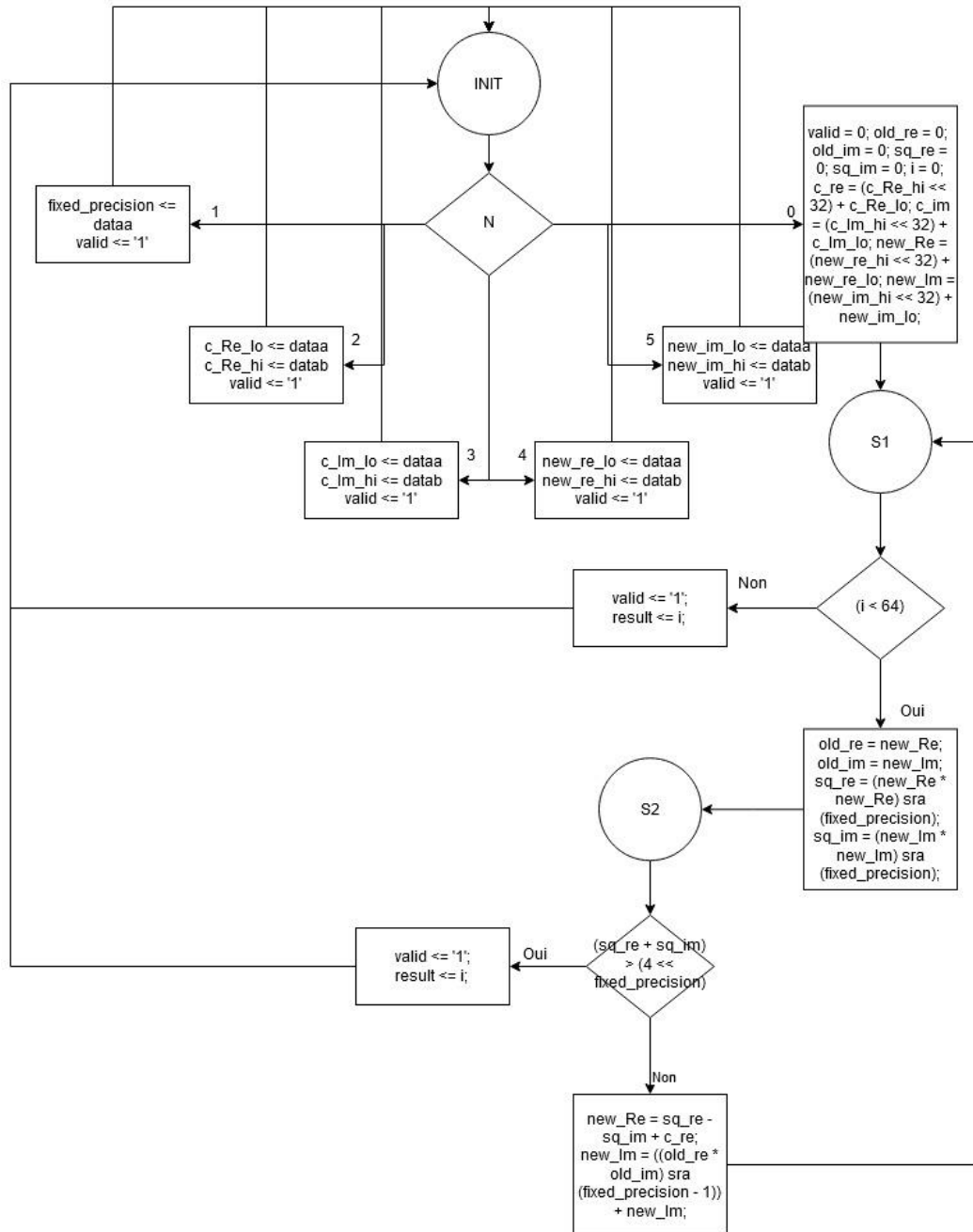


Figure 2 : Schéma pour l'instruction multicycle et machine à état 1.

Preuve d'accélération

Pour mesurer l'accélération de notre application, il sera question pour nous de considérer tout d'abord le nombre d'appels à la fonction `get_fractal_level()`. Celui-ci peut être obtenu à partir du profilage de l'application dans eclipse, par la suite, nous devrons considérer le nombre d'instructions assembleur qui composent la fonction `get_fractal_level()` et en faisant l'approximation d'un cycle d'horloge nécessaire pour réaliser une instruction assembleur, nous pourrions ainsi obtenir le temps total nécessaire pour l'exécution de la fonction en c.

Par la suite, nous identifierons le temps nécessaire à l'exécution de la fonction mais cette fois-ci lorsqu'elle est sous -forme de machine à état. Dans ce cas, nous considérerons un cycle d'horloge par état. Il est à noter qu'il faudra dans les 2 cas considérer la divergence après une même valeur de i. ici, nous prendrons i = 1 pour simplifier l'analyse du code assembleur.

Pour i = 1, chaque instruction assembleur est exécutée approximativement une fois ce qui nous donne sensiblement 170 instructions assembleur, de plus on considèrera que les instructions de type `call` s'exécute en un cycle d'horloge. En nous basant sur le profilage, le nombre d'appel à la fonction `get_fractal_level()` est : 306081 ce qui nous donne approximativement 52033770 cycles d'horloge pour une divergence après une itération.

3.2.2 Instruction spécialisée 2 (Michael Flandez)

L'ASM 2 sera une instruction qu'écrira sur le bus Avalon vers l'écran VGA.

Code source

```

unsigned int set_pixel_ASM_2(unsigned int n, unsigned int dataaa, unsigned int datab) { // pas optimise
    int start = 1; // Not really used, just for simulation purpose in C/C++
    int valid = 0; // Not really used, just for simulation purpose in C/C++
    int wr = 0; // Not really used, just for simulation purpose in C/C++
    static unsigned int data = 0; // Not really used, just for simulation purpose in C/C++
    int busy = 0; // Not really used, just for simulation purpose in C/C++
    static unsigned int result; //All registers must be declared static to keep their value from call to call
    static unsigned int VGA_local; //idem
    static unsigned int color; //idem
    static unsigned int X_pixel; //idem
    static unsigned int Y_pixel; //idem
    static unsigned int dx; //idem
    static unsigned int dy; //idem

    INIT: if ((start == 0) && (busy == 0)) { valid = 0; goto INIT; }
    else if ((start == 1) && (n == 0) && (busy == 0)) { valid = 0; VGA_local = dataaa - 614436; color = datab; result = VGA_local; return result; }
    else if ((start == 1) && (n == 1) && (busy == 0)) { valid = 0; X_pixel = dataaa; Y_pixel = datab; result = dataaa; return result; }
    else if ((start == 1) && (n == 2) && (busy == 0)) { valid = 0; dx = dataaa; dy = datab; result = dataaa; goto S1; }
    else if ((start == 1) && (n == 3) && (busy == 0)) { valid = 0; goto S2; }
    else if ((start == 1) && (n == 4) && (busy == 0)) { valid = 0; goto S3; }
    else if ((start == 1) && (n == 5) && (busy == 0)) { valid = 0; goto S4; }
    else if ((start == 1) && (n == 6) && (busy == 0)) { valid = 0; goto S5; }
    else if ((start == 1) && (n == 7) && (busy == 0)) { valid = 0; goto S6; }
    else if ((start == 1) && (n == 8) && (busy == 0)) { valid = 0; goto S7; }
    else if ((start == 1) && (n == 9) && (busy == 0)) { valid = 0; goto S8; }
    else if ((start == 1) && (n == 10) && (busy == 0)) { valid = 0; goto S9; }
    else { valid = 0; goto INIT; }

    END: if (!wr) { wr = 0; valid = 1; return result; }
    S1: if (!wr) { result = VGA_local + (X_pixel + dx) + 640 * (Y_pixel + dy); data = color; wr = 1; goto END; } // Y_pixel<<9 + Y_pixel<<7
    S2: if (!wr) { result = VGA_local + (X_pixel + dx) + 640 * (Y_pixel - dy); data = color; wr = 1; goto END; }
    S3: if (!wr) { result = VGA_local + (X_pixel - dx) + 640 * (Y_pixel + dy); data = color; wr = 1; goto END; }
    S4: if (!wr) { result = VGA_local + (X_pixel - dx) + 640 * (Y_pixel - dy); data = color; wr = 1; goto END; }
    S5: if (!wr) { result = VGA_local + (X_pixel + dy) + 640 * (Y_pixel + dx); data = color; wr = 1; goto END; }
    S6: if (!wr) { result = VGA_local + (X_pixel + dy) + 640 * (Y_pixel - dx); data = color; wr = 1; goto END; }
    S7: if (!wr) { result = VGA_local + (X_pixel - dy) + 640 * (Y_pixel + dx); data = color; wr = 1; goto END; }
    S8: if (!wr) { result = VGA_local + (X_pixel - dy) + 640 * (Y_pixel - dx); data = color; wr = 1; goto END; }
    S9: if (!wr) { result = VGA_local + (X_pixel) + 640 * (Y_pixel); data = color; wr = 1; goto END; }
}

```

Figure 3 : Code source pour l'instruction multicycle qui prends trois paramètres unsigned int et écrit des données sur le bus Avalon vers le module VGA.

Rapport – Fractale

Cette fonction est appelée plusieurs fois dans le code, dans `draw_balls()` et dans `draw_fractal()` et fait des opérations spécifiques pour des différents valeurs de N. Lorsque N est zéro, on garde la valeur du deuxième paramètre dans la variable `VGA_Local` qui est l'adresse de l'écran vga, le troisième paramètre 'color' est stocké dans la variable du même nom.

En N égale à un, on est dans l'appel dans la fonction de `draw_balls()`, on reçoit et garde les valeurs de la position du centre de cercle sur deux dimensions X et Y du pixel à dessiner. En N égale à deux, on reçoit deux nouveaux paramètres 'dx' et 'dy' qui sont utilisés pour calculer les positions des pixels à dessiner autour du centre du cercle. Pour les N allant dès 3 jusqu'à 9 on ne passe pas de paramètres mais on calcule les pixels restants.

En N égale à 10, on fait appel à la fonction dès la fonction `draw_fractal()`, et là on passe les paramètres X et Y du pixel à dessiner.

Schéma

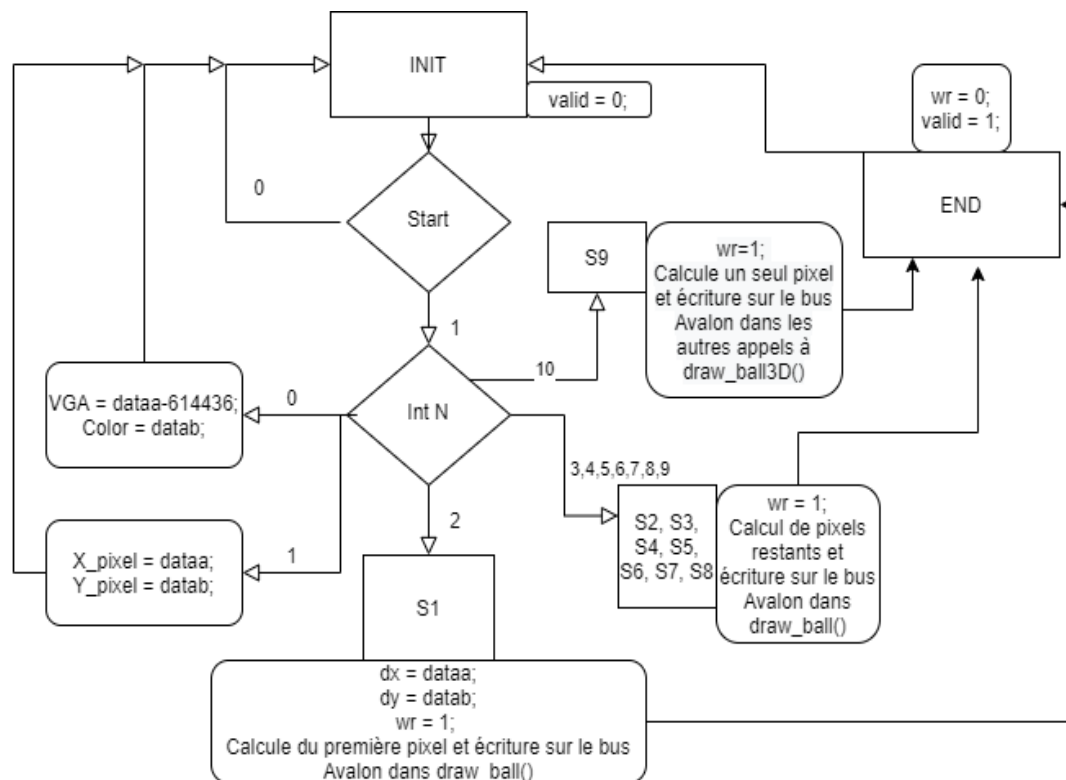


Figure 4 : Schéma pour l'instruction multicycle et machine à état 2.

Sur cette image, on voit que si start est à 0 on reste dans l'état INIT, sinon on analyse le paramètre en entrée N. Le code C doit faire appel à l'instruction en N égale à 0 pour avoir l'adresse en mémoire de l'écran VGA. Ensuite le code permettra de dessiner des pixels simples en utilisant la fonction spécialisée en N égale à 10 ou sinon un groupe de huit pixels à dessiner en utilisant les instructions spécialisées en N égale à 1, 2, 3, 4, 5, 6, 7, 8, 9.

Dans le code C, on dessine les pixels un à la fois en appelant les fonctions séquentiellement en N égale à 2, 3, 4, 5, 6, 7, 8, 9, mais en VHDL, on utilisera une seule instruction qui fera passer par tous les étapes automatiquement sans la nécessité de faire plusieurs appels.

Simulation logiciel

Le code a été développé de façon à avoir des retours des calculs dans la console de Visual Studio. En mode débogage, on est capable de vérifier le bon passage des paramètres, ainsi que les calculs correspondants qui seront plus utiles lors de l'implémentation sur VHDL.

Preuve d'accélération

On trouve sur le tableau suivant le nombre d'appels totales fait à la fonction `set_pixel_color()`.

					Nombre d'appels
draw_ball3D()					2616
draw_fractal()					307200
VGA::clr()					307200
	VGA::Set_Pixel_Color	total	0.95		617016

Tableau 3 : Liste de fonctions qu'appellent les fonctions `Set_Pixel_Color`.

Sur le tableau 3, on voit en couleur verte les trois fonctions qui font appel à `Set_Pixel_Color`. `Draw_ball3D` l'appelle 2616 fois, `draw_fractal()` 307200 fois et `VGA::clr()` 307200 fois.

On a également la liste d'instructions en code assembleur de deux parties.

- 1) `Set_Pixel_Color()` utilise 63 instructions en code assembleur pour sa seule.
- 2) Dans la fonction `draw_ball3D()` présenté à continuation :

```
int xhi = x + dx;
int xlo = x - dx;
int yhi = y + dy;
int ylo = y - dy;

vga.Set_Pixel_Color(xhi,yhi,color);
vga.Set_Pixel_Color(xhi,ylo,color);
vga.Set_Pixel_Color(xlo,yhi,color);
vga.Set_Pixel_Color(xlo,ylo,color);

xhi = x + dy;
xlo = x - dy;
yhi = y + dx;
ylo = y - dx;

vga.Set_Pixel_Color(xhi,yhi,color);
vga.Set_Pixel_Color(xhi,ylo,color);
vga.Set_Pixel_Color(xlo,yhi,color);
vga.Set_Pixel_Color(xlo,ylo,color);
```

On a également des instructions en code assembleur. Le total pour cette partie est de 103 instructions.

Donc, pour la fonction `draw_ball3D()` on aura :

$$\text{nombre d'instructions total} = \left(\frac{2616}{8}\right) * (103 + (8 * 63)) = 327 * 607$$

$$\text{total} = 198\,489 \text{ instructions approximativement}$$

Pour les autres appels on aura :

$$\text{nombre total d'instructions} = (307200 * 63) * 2 = 38\,707\,200 \text{ instructions}$$

4 Évaluation des performances

4.1 Accélération attendues

4.1.1 ASM 1 (Sekou Kone)

Dans le cadre de la machine à état, nous notons que l'on a d'abord pour chaque appel de la fonction 5 cycles d'horloge pendant lesquels on initialise les paramètres puis nous avons 5 autres cycles d'horloge pendant lesquels l'algorithme s'exécute jusqu'à obtenir une divergence après une itération. Ce qui nous fait donc un total de : 3060810 cycles d'horloge soit une accélération de 17, en considérant que la fonction `get_fractal_level()` dans tous ses appels consomme en c un temps de 17.55 s nous aurons au final gagné approximativement 16 secondes.

4.1.2 ASM 2 (Michael Flandez)

En considérant une instruction par cycle d'horloge, ou bien un état par instruction, on voit que selon l'ASM 2 le nombre d'instructions sera :

`Draw_ball3D()` :

Pour l'instruction 0 et 1, on a une instruction.

Pour l'instruction 2 à 9, on a 3 instructions.

$$\text{Nombre d'instructions dans la fonction} = 8 * 3 + 2 = 26$$

Si on prend le cas de 2616 appels, ceci nous donne :

$$\text{nombre d'instructions total} = \left(\frac{2616}{8}\right) * (26) = 327 * 26 = 8502$$

Ceci équivaut à une réduction de temps de :

$$\% \text{ réduction} = 1 - \left(\frac{8507}{198489}\right) = 95.7\%$$

1) `VGA ::clr()` et `draw_fractal()` :

Rapport – Fractale

Pour l'instruction 0 et 10, on a un et trois instructions respectivement.

$$\text{Nombre d'instructions} = 1 + 3 = 4$$

De la même manière :

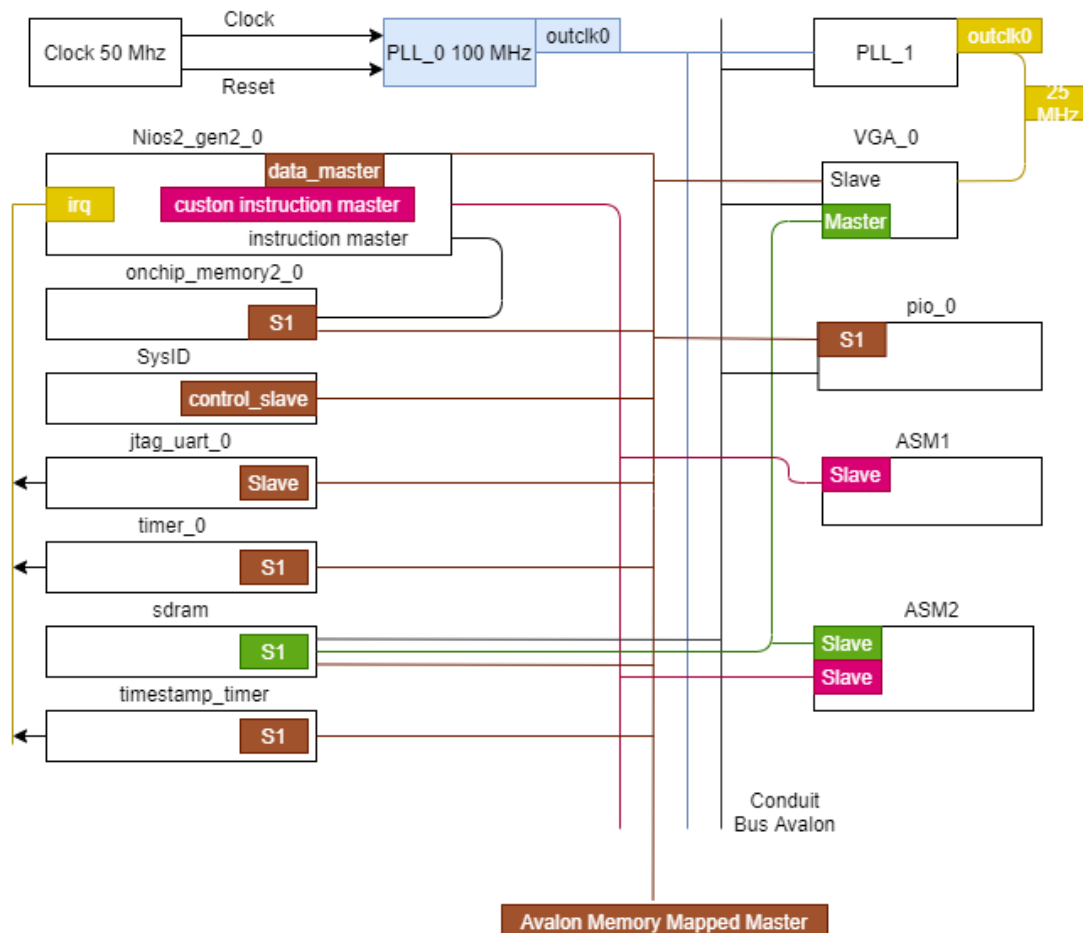
$$\text{nombre total d'instructions} = (307200 * 4) * 2 = 2\,457\,600 \text{ instructions}$$

$$\% \text{ réduction} = 1 - \left(\frac{2\,457\,600}{38\,707\,200} \right) = 93.65\%$$

De manière générale, selon la contribution des deux fonctions, on peut s'attendre à une amélioration de :

$$\% \text{ réduction} = 1 - \left(\frac{2\,457\,600 + 8507}{38\,707\,200 + 198489} \right) = 93.66\%$$

4.2 L'architecture générale



Tous les modules sont branchés aux signaux clock et reset de PLL_0, sauf VGA qui est connecté à PLL_1
Ces signaux n'ont pas été dessinés par propriété

Figure 5 : Schéma de l'architecture générale de la partie matériel en implémentant les deux ASM en VHDL.

On peut voir sur la figure 5, l'architecture générale de la partie matérielle ainsi que les connexions entre les différents modules par rapport aux adresses mémoires de chaque composant, par rapport aux instructions spécialisés (custom instruction master) et par rapport au conduit du bus Avalon ('wire').

Si bien on voit un fil 'outclk0' sortir du PLL_0, on le voit que ce fil n'est pas connecté. En vérité, il est branché à tous les composants sauf à VGA_0. Il rentre dans les entrées 'clock_input' et 'reset_input' qui ne sont pas affichés afin de garder la lisibilité du schéma.

- Clock 50 MHz: horloge maître à 50 MHz. Clock envoie l'horloge à PLL_0, et reset à tous les modules.
- PLL_0 : horloge esclave à l'horloge Clock_50. Sa vitesse est de 100 MHz avec un petit déphasage de -3800 ps. C'est cette horloge qui donne la cadence à tous les composants du système sauf à l'écran VGA_0 qui est esclave de PLL_1. PLL_0 peut envoyer des 'resets' également.
- PLL_1 : horloge qui tourne à environ 25 MHz, esclave de l'horloge PLL_0, chargé d'envoyer l'horloge à VGA.
- Nios2 : processeur matériel. Gère les instructions, les appels à des instructions et à des instructions spécialisées, les accès mémoires, les interruptions, les 'resets' de débogage.
- Onchip_memory : mémoire rapide sur le chip de 128 ko, connecté au data master et à PLL_0.
- SysId : périphérique permettant d'identifier le système matériel.
- Jtag_uart_0 : lien USB.
- Timer_0 : compteur de 32 bits de 1ms.
- Sdram : contrôleur. Mémoire de 64 MB où sont stockés les instructions.
- Timestamp_timer : deuxième compteur de 32 bits de 1us pour le profileur GNU.
- VGA_0 : Écran périphérique, avec une adresse mémoire, connecté à la SDRAM comme maître.
- Pio_0 : périphérique clavier d'entrée avec des boutons.
- ASM_1 et ASM_2 : instructions spécialisées.

On remarque qu'en plus de l'architecture de base, les deux nouvelles instructions spécialisées sont présentées. Les deux ASM 1 sont branchés en mode esclave à 'custom instruction master' du processeur Nios2 et ils reçoivent l'horloge de 100 MHz provenant du PLL_0. L'ASM est aussi branché en mode esclave au contrôleur SDRAM, laquelle est aussi connectée au VGA qui est en fonction maître.

5 Plan de tests

5.1 ASM 1 (Sekout Kone)

Plan de test pour l'asm :

Pour pouvoir tester l'asm de l'instruction multicycle correspondant à la fonction get_fractal_level, après l'avoir implémenté en VHDL, nous allons l'appeler dans le code c avec les différentes valeurs de n pour vérifier son comportement adéquat. Pour rappel, les valeurs de n correspondent aux actions suivantes :

N = 1 : sauvegarde de fixed_precision provenant de dataa.

N = 2 : sauvegarde de c_Re_lo et de c_Re_hi provenant respectivement de dataa et datab.

N = 3 : sauvegarde de c_Im_lo et de c_Im_hi provenant respectivement de dataa et datab.

N = 4 : sauvegarde de new_re_lo et de new_re_hi provenant respectivement de dataa et datab.

N = 5 : sauvegarde de new_im_lo et de new_im_hi provenant respectivement de dataa et datab.

N = 0 : initialisation des paramètres et concaténation des registres, puis exécution de l'algorithme get_fractal_level().

Pour tester le fonctionnement de notre exécution nous allons initialiser les variables à des valeurs connues et vérifier que le nombre d'itérations est bien correct en fin d'algorithme, nous aurons donc :

- N = 1 : sauvegarde de fixed_precision à la valeur 26.
- N = 2 : sauvegarde de c_Re_lo et de c_Re_hi respectivement à 4242622383 et à 4294967295.
- N = 3 : sauvegarde de c_Im_lo et de c_Im_hi respectivement à 10603200 et à 0.
- N = 4 : sauvegarde de new_re_lo et de new_re_hi respectivement à 4205488811 et à 4294967295.
- N = 5 : sauvegarde de new_im_lo et de new_im_hi respectivement à 4227187344 et à 4294967295.
- N = 0 : On s'attend à recevoir en sortie une valeur de i = 1.

5.2 ASM 2 (Michael Flandez)

Les tests seront faits de manière séquentielle.

- 1) Vérifier que l'adresse du VGA est correctement passé dans l'instruction.
- 2) Vérifier que tous les autres paramètres sont bien passés.
- 3) Vérifier que tous les calculs, donc tous les résultats sont cohérents.
- 4) Vérifier que tous les cercles sont bien dessinés sur l'écran en temps normal.
- 5) Modifier le code de façon à pouvoir intégrer cette instruction spécialisée dans les autres parties du code où il est pertinent de rajouter.
- 6) Vérifier le bon fonctionnement de ses ajoutes.
- 7) Procéder à des améliorations ou à des optimisations du code.
- 8) Vérifier le correcte fonctionnement des optimisations

6 Conclusion

Dans ce document on a détaillé une méthode d'optimisation d'un projet original dessinant un fractale avec 10 perles. En premier temps, on a remarqué que les vitesses d'actualisation de l'image étaient sous un seuil acceptable. Il faudrait que l'image s'actualise à une vitesse plus rapide de façon à donner l'impression à l'œil de continuité.

En faisant des profilages du code, on a trouvé les points demandant le plus de ressources. De là, on a passé à optimiser le code en C, et ensuite à dessiner des emplacements pour créer des instructions spécialisées. On en a implémenté deux pour, en forme de machines à états pour simuler leur fonctionnalité.

Dans la prochaine partie du projet, ces instructions seront implémentées directement en matériel où on pourra faire accélérer les calculs grâce à l'efficacité du parallélisme en VHDL.

Il faut remarquer que ces avances pourraient changer grâce à des optimisations continues.

A.1 Annexe

1. <https://stackoverflow.com/questions/922599/is-it-still-worth-trying-to-create-optimizations-for-sqrt-in-c> Ressource trouvé qu'optimise et accélère le calcul de la racine carrée en utilisant une interpolation.